House Price Prediction Using Machine Learning: An End-to-End Regression Study

Nidamanuru Shanmukha Ram
Department of Artificial Intelligence and Machine Learning
Vignan's Foundation for Science, Technology and Research
Bapatla, India

***Abstract*—This paper presents an end-to-end machine learning system for predicting residential house sale prices using the Kaggle House Prices: Advanced Regression Techniques dataset. The project treats house price prediction as a supervised regression problem and covers data understanding, exploratory data analysis, missing-value treatment, outlier removal, feature engineering, target transformation, categorical encoding, feature scaling, model development, five-fold cross-validation, hyperparameter tuning, model comparison, feature-importance analysis, and prediction on unseen test data. Seven regression algorithms are evaluated. XGBoost achieves the lowest mean five-fold cross-validation RMSE of 0.11797 and is selected as the final model.**

***Index Terms*—Machine Learning, House Price Prediction, Regression, Ames Housing Dataset, Feature Engineering, Cross-Validation, XGBoost, Model Evaluation**

I. Introduction

Machine learning provides computational methods for learning patterns from historical data and using them to make predictions on unseen observations. House price prediction is a practical regression problem because property prices depend on interacting characteristics such as quality, area, age, neighborhood, garage, and basement attributes. This project develops a complete machine learning pipeline for predicting residential property prices in Ames, Iowa.

II. Problem Statement

The objective is to develop a regression system capable of predicting the final sale price of residential houses from structural, quality, location, and other property-related characteristics. The target variable is `SalePrice` and the dataset provides 80 predictive features.

III. Research Objectives

The objectives are to understand the dataset, perform EDA, handle missing values and outliers, engineer meaningful features, transform skewed variables, encode categorical variables, scale features, develop multiple regression models, validate and compare them using five-fold cross-validation, tune selected models, select the best model, analyze important features, and generate predictions and a Kaggle submission.

IV. Research Questions

The study investigates: (1) which housing characteristics are strongly related to `SalePrice`; (2) how preprocessing affects the dataset; (3) whether feature engineering improves representation; (4) which regression algorithm performs best; (5) how consistently models perform under cross-validation; (6) whether tuning improves performance; (7) which features contribute most to predictions; and (8) whether the final model predicts unseen houses effectively.

V. Literature Review / Related Work

House price prediction has been addressed using statistical regression and machine learning. Ridge and Lasso introduce L2 and L1 regularization respectively, while Elastic Net combines both. Tree ensembles such as Random Forest and Gradient Boosting are effective for structured tabular data. XGBoost and LightGBM provide efficient gradient-boosting approaches for prediction tasks. The present work combines regularized linear models and ensemble methods in one evaluated pipeline.

VI. Research Gap

A basic model trained directly on raw variables may not adequately handle missing values, skewness, categorical variables, outliers, and nonlinear relationships. This project addresses these issues through domain-based preprocessing, feature engineering, transformations, robust scaling, multiple algorithms, cross-validation, tuning, and feature-importance analysis.

VII. Dataset Description

The project uses the Kaggle House Prices: Advanced Regression Techniques dataset based on residential properties in Ames, Iowa. The training set contains 1460 observations and 81 columns, comprising 80 input features and the `SalePrice` target. The test set contains 1459 observations and 80 input features without the target.

VIII. Methodology

The methodology consists of dataset loading, understanding, EDA, missing-value analysis, target transformation, outlier detection and removal, imputation, feature engineering, skewness transformation, categorical encoding, scaling, model development, five-fold cross-validation, hyperparameter tuning, final model selection, feature-importance analysis, unseen-data prediction, and submission generation.

IX. System / ML Workflow

Dataset → Data Loading → EDA → Preprocessing → Feature Engineering → Transformation → Encoding → Scaling → Model Training → Cross-Validation → Tuning → Model Selection → Prediction → Submission

X. Data Preprocessing

The identifier `Id` is removed and `SalePrice` is separated from the features. The target is transformed using $\log(1+y)$. Domain-based missing-value handling is applied: absent facilities are represented by "None" or zero where appropriate, while `LotFrontage` is imputed using neighborhood median values. An influential `GrLivArea`/price outlier condition is also removed. Numerical features are transformed when sufficiently skewed, categorical variables are label encoded, and RobustScaler is applied.

XI. Exploratory Data Analysis (EDA)

EDA includes target distribution analysis, log-transformed target distribution, missing-value analysis, correlation analysis, feature-versus-target plots, and outlier inspection. Strong relationships are observed for variables such as `OverallQual`, `GrLivArea`, `GarageCars`, and `GarageArea`.

XII. Feature Engineering / Feature Selection

Engineered variables include total square footage, total bathrooms, total porch area, house age, remodeling age, binary indicators for the presence of selected facilities, and an interaction feature `QualSF = OverallQual × TotalSF`. Numerical skewness above 0.75 in absolute value is addressed with `boxcox1p` using $\lambda = 0.15$. Feature importance is later used to analyze influential predictors.

XIII. Model Development

Seven regression algorithms are developed: Ridge Regression, Lasso Regression, Elastic Net, Random Forest Regression, Gradient Boosting Regression, XGBoost Regression, and LightGBM Regression. Regularized linear models provide controlled linear baselines, while tree-based ensembles capture nonlinear relationships.

XIV. Experimental Setup

The implementation uses Python in Jupyter Notebook/VS Code with Pandas, NumPy, Matplotlib, Seaborn, SciPy, Scikit-learn, XGBoost, and LightGBM. The random seed is 42. Five-fold shuffled cross-validation is used, and RMSE is the primary comparison metric on the log-transformed target.

XV. Model Validation

Five-fold cross-validation divides the training data into five folds. Four folds are used for training and one for validation in each iteration, with every fold serving as validation once. Mean and standard deviation of RMSE provide an estimate of predictive performance and stability. Lower RMSE is better.

XVI. Hyperparameter Tuning

GridSearchCV is used for selected models. Ridge evaluates several α values; Lasso evaluates several regularization strengths; and Random Forest evaluates combinations of estimator count, maximum depth, and minimum split size. Five-fold validation is retained during tuning.

XVII. Model Evaluation

The main evaluation metric is RMSE:

$$RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}.$$

MAE and R^2 are also used for the final comparison. Lower RMSE and MAE indicate smaller errors, while higher R^2 indicates greater explained variance.

XVIII. Model Comparison

Table I reports the actual five-fold cross-validation results obtained from the executed project notebook.

TABLE I
Five-Fold Cross-Validation Results

Model	Mean RMSE	Std. RMSE
Ridge	0.11925	0.00867
Lasso	0.11870	0.00915
Elastic Net	0.11870	0.00916
Random Forest	0.13439	0.00772
Gradient Boosting	0.12333	0.00692
XGBoost	**0.11797**	**0.00483**
LightGBM	0.12324	0.00833

XIX. Final Model Selection

XGBoost is selected because it achieves the lowest mean cross-validation RMSE, 0.11797, among the evaluated models. Its standard deviation of 0.00483 also indicates relatively stable cross-validation performance.

XX. Sample / Unseen Data Prediction

The selected model is trained on the processed training data and used to predict the Kaggle test dataset. Since the target was log-transformed, predictions are returned to the original price scale using $\exp(\hat{y}) - 1$. The project generates a comparison of sample test predictions and creates the required submission file.

XXI. Results

The executed notebook reports the following final comparison values: MAE = 0.08081, MSE = 0.01394, RMSE = 0.11797, and R^2 = 0.91215. The training evaluation reports RMSE of approximately \$13,617.59 and R^2 of approximately 0.9706. The final Kaggle prediction file is `submission.csv`.

XXII. Discussion

The results show that preprocessing and feature engineering are important for this structured regression task. XGBoost provides the best cross-validation RMSE among the seven evaluated approaches. The combination of quality, area, age, garage, basement, and engineered aggregate features gives the models useful representations of house characteristics. Cross-validation provides a more reliable comparison than a single split.

XXIII. Limitations

The dataset represents properties in Ames, Iowa, so the model may not generalize directly to other regions. Housing markets change over time, and factors such as interest rates and broader economic conditions are not directly represented. Label encoding also imposes numerical representations on categorical values. Model performance depends on the preprocessing and experimental configuration used in the project.

XXIV. Conclusion

This study developed a complete end-to-end machine learning workflow for house price prediction. The pipeline included EDA, missing-value treatment, outlier handling, feature engineering, target and feature transformations, encoding, scaling, model development, five-fold cross-validation, hyperparameter tuning, model comparison, and unseen-data prediction. Seven regression algorithms were evaluated, with XGBoost obtaining the lowest mean cross-validation RMSE of 0.11797. The project demonstrates a practical and reproducible approach to structured regression.

XXV. Future Scope

Future work can explore more extensive hyperparameter optimization, stacking and blending, alternative categorical encoders, advanced feature selection, SHAP-based explainability, detailed error analysis, deployment through Streamlit or FastAPI, and monitoring of model performance after deployment.

XXVI. References

References

[1] D. De Cock, "Ames, Iowa: Alternative to the Boston Housing Data as an End of Semester Regression Project," Journal of Statistics Education, vol. 19, no. 3, 2011.

[2] Kaggle, "House Prices: Advanced Regression Techniques." [Online]. Available: https://www.kaggle.com/competitions/house-prices-advanced-regression-techniques

[3] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[4] L. Breiman, "Random Forests," Machine Learning, vol. 45, pp. 5–32, 2001.

[5] J. H. Friedman, "Greedy Function Approximation: A Gradient Boosting Machine," The Annals of Statistics, vol. 29, no. 5, pp. 1189–1232, 2001.

[6] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," Proc. 22nd ACM SIGKDD, pp. 785–794, 2016.

[7] G. Ke et al., "LightGBM: A Highly Efficient Gradient Boosting Decision Tree," NeurIPS, vol. 30, 2017.

[8] G. E. P. Box and D. R. Cox, "An Analysis of Transformations," Journal of the Royal Statistical Society: Series B, vol. 26, no. 2, pp. 211–243, 1964.

[9] G. James, D. Witten, T. Hastie, and R. Tibshirani, An Introduction to Statistical Learning, Springer, 2013.

[10] A. Geron, Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow, 3rd ed., O'Reilly Media, 2022.

Appendix A
Appendix

The implementation was developed in the Jupyter notebook `house_price_project_separated (1).ipynb`. The project produces EDA visualizations, cross-validation comparison, feature-importance analysis, and `submission.csv`. The reported numerical results in this paper are taken from the executed notebook.